



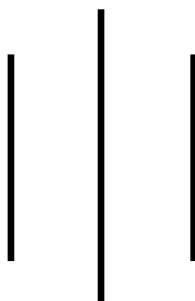
**LA GRANDEE INTERNATIONAL COLLEGE**

**Simalchaur, Pokhara Nepal**

Final Project

On

**“Banking System”**



**Submitted to:**

Bachelor of Computer Application (BCA) Program

In partial fulfilment of the requirements for the degree of BCA under

Pokhara University

**Submitted by:**

| <b>Name:</b>    | <b>Course</b> | <b>Semester</b> | <b>P.U. Registration Number</b> |
|-----------------|---------------|-----------------|---------------------------------|
| Anish Giri      | BCA           | 2 <sup>nd</sup> | 2025-1-53-0314                  |
| Ankit S. Paudel | BCA           | 2 <sup>nd</sup> | 2025-1-53-0315                  |
| Khagaraj Poudel | BCA           | 2 <sup>nd</sup> | 2025-1-53-0327                  |

**Date:16/07/2026**

## Acknowledgement

The satisfaction that accompanies after the successful completion of any task will be incomplete without mentioning the people whose ceaseless and relentless cooperation, constant guidance and encouragement made this project possible.

We are grateful to our project supervisor **Mr Arpan Pokhrel**, our faculty teacher **Mr. Ramesh Chalise**, for the guidance, inspiration and constructive suggestions that helped us in the preparation of this project.

We are also appreciative among each other and have understood that teamwork, the designation of the task per the skillset one portrays, constant synchronisation and monitoring of progress and instilling new knowledge and skill is imperative for the success of any given work.

Sincerely,

Anish Giri (25530410)

Ankit Sharma Paudel (25530411)

Khagaraj Poudel (25530421)

**Declaration for**  
**“Banking System”**  
**Student’s Declaration**

We hereby declare that we are the only authors of this work and that no sources other than the listed here have been used in this work.

Anish Giri (25530410)

Ankit Sharma Paudel (25530411)

Khagaraj Poudel (25530421)

BCA, 2<sup>nd</sup> semester

Date: 12/07/2026

### **Supervisor's Declaration**

I hereby recommend that this project entitled “**Banking System**” is done under my supervision by **Anish Giri, Ankit Sharma Paudel and Khagaraj Poudel**, during their second Semester in partial fulfilment of the requirements for the degree of **BCA** under **Pokhara University** is completed to my satisfaction and be processed for final evaluation.

---

Arpan Pokhrel

Date: 12/07/2026

## Letter of Approval

We certify that we have examined this report entitled “**Banking System**” and are satisfied with the project defence. In our opinion it is satisfactory in the scope and qualify as project in partial fulfilment of the requirements for the degree of **BCA** under **Pokhara University**.

---

Ankit Poudyal  
Program  
Coordinator

---

Er. Dikshan Gurung  
Examiner

---

Er. Kiran KC  
Principal

Date: 12/07/2026

## Table of Contents

|                                           |    |
|-------------------------------------------|----|
| 1. Introduction .....                     | 1  |
| 2. Problem Statement .....                | 3  |
| 3. Objectives .....                       | 4  |
| 4. Background Study .....                 | 5  |
| 5. Requirement Document.....              | 6  |
| 5.1 Types of Requirements .....           | 6  |
| 5.2 Requirement Matrix: .....             | 7  |
| 6. System Design.....                     | 8  |
| 6.1 Gantt Chart.....                      | 8  |
| 6.2 Use Case Diagram.....                 | 9  |
| 6.3 Data Flow Diagram.....                | 10 |
| 6.4 Flowchart .....                       | 12 |
| 7. Development .....                      | 14 |
| 7.1 Development Model.....                | 14 |
| 7.2 Tools and Collaboration Platform..... | 14 |
| 7.3 Work Assignment .....                 | 14 |
| 8. Testing.....                           | 15 |
| 8.1 Example of a Test Case.....           | 16 |
| 8.2 Test Matrix of Tested Modules.....    | 17 |
| 8.3 Requirement Traceability Matrix.....  | 18 |
| 8.4 User Acceptance Testing .....         | 18 |
| 9. Project Results.....                   | 19 |
| 10. Conclusion .....                      | 20 |
| 11. References .....                      | 22 |
| 12. Annexures .....                       | 23 |

## LIST OF FIGURES

|                                            |    |
|--------------------------------------------|----|
| Figure 1: Gantt Chart .....                | 8  |
| Figure 2: Use Case Diagram .....           | 9  |
| Figure 3: Data Flow Diagram Level 0 .....  | 10 |
| Figure 4: Data Flow Diagram Level 1 .....  | 11 |
| Figure 5: Flowchart of Simple Banking..... | 12 |
| Figure 6: Flowchart of Dashboard .....     | 13 |
| Figure 7: Login and Signup .....           | 23 |
| Figure 8: Account Creation.....            | 23 |
| Figure 9: Account login .....              | 24 |
| Figure 10: User Menu .....                 | 24 |
| Figure 11: Add balance .....               | 25 |
| Figure 12: Check Balance .....             | 25 |
| Figure 13: Transaction History .....       | 25 |
| Figure 14: Change Password .....           | 26 |
| Figure 15: Delete Account .....            | 26 |
| Figure 16: Account Logout.....             | 27 |

## LIST OF TABLES

|                                                  |    |
|--------------------------------------------------|----|
| Table 5.1:Requirement Matrix .....               | 7  |
| Table 7.1: Work Assignment Table .....           | 14 |
| Table 8.1: Module wise Test Cases .....          | 17 |
| Table 8.2: Requirement Traceability Matrix ..... | 18 |

## ABBREVIATIONS

| Abbreviation | Full Form                        |
|--------------|----------------------------------|
| BCA          | Bachelor of Computer Application |
| CLI          | Command Line Interface           |
| DFD          | Data Flow Diagram                |
| I/O          | Input / Output                   |
| P.U.         | Pokhara University               |
| RTM          | Requirement Traceability Matrix  |
| SDLC         | Software Development Life Cycle  |
| SRD          | Software Requirement Document    |
| STLC         | Software Testing Life Cycle      |
| TID          | Test Case Identifier             |
| UAT          | User Acceptance Testing          |

# **1. Introduction**

The Banking Management System is a computerized application designed to enable customers to perform essential banking activities such as account creation, deposits, withdrawals, fund transfers, and balance inquiries, all while securely maintaining customer records. In today's fast-paced world, banking has become an integral part of everyday life, and the demand for quick, reliable, and secure financial services has increased significantly. Keeping this in mind, the main objective of this project is to provide a convenient, efficient, and secure method for managing banking operations, reducing the dependency on manual processes and human intervention.

The traditional banking system relies heavily on manual record-keeping, which often results in delays, data redundancy, security risks, and difficulties in managing customer information. In a manual system, maintaining ledgers, registers, and paper-based records makes it difficult to update information in real time, increases the chances of human error, and makes it nearly impossible to ensure complete data security. Retrieving records and processing transactions can be time-consuming and less reliable, especially as the number of customers and transactions grows. Additionally, manual systems make it difficult to generate accurate reports, track transaction histories, and detect fraudulent activities in a timely manner.

To overcome these limitations, we developed a computerized Banking Management System using the C programming language. This system was designed to automate the core functions of banking, ensuring that customer information and transactions are handled in a structured, accurate, and secure manner. By using file handling and structured programming concepts in C, the system is able to store, retrieve, and update customer data efficiently without the drawbacks associated with paper-based methods.

The project work was divided among team members, including documentation, system design, research, coding, and testing, ensuring that each aspect of the project received proper attention and expertise. During development, we faced several challenges related to customer data management, transaction processing, data accuracy, and system security. Overcoming these challenges required careful planning, logical problem-solving, and continuous testing and debugging. However, these challenges

ultimately helped us improve the quality, reliability, and functionality of the system, while also enhancing our understanding of software development practices.

The developed system reduces paperwork, improves data accuracy and security, speeds up transaction processing, and simplifies record management. It also minimizes the possibility of errors that commonly occur in manual systems and ensures that sensitive customer information remains protected. As a result, the Banking Management System enhances the overall efficiency of banking operations and provides better, faster, and more reliable services for both customers and bank employees, making it a valuable step towards modernizing the banking sector.

## **2. Problem Statement**

As this was our first programming-based project, we faced several challenges during the development process. Since none of us had prior experience working on a complete software project of this scale, many errors and technical issues arose while designing and implementing the system. There were times when we had to revisit our code, correct logical errors, and rework certain modules to make the system function properly. However, overcoming these difficulties helped us gain valuable practical experience, strengthen our programming and problem-solving skills, and significantly improve the overall quality of the project.

The existing banking system relied heavily on manual record-keeping, which often led to data duplication, security risks, excessive paperwork, and delays in processing transactions. Managing customer information and maintaining records were time-consuming tasks that required constant manual effort, making the system less efficient, more prone to errors, and difficult to scale as the number of customers increased. Employees often had to spend a considerable amount of time searching through physical files just to retrieve basic customer details, which further slowed down day today banking operations. The major issues identified were:

1. Data redundancy and paperwork
2. Security and record management problems
3. Slow and outdated manual processes
4. Lack of accuracy in transactions
5. Difficulty in generating reports and tracking history

To overcome these challenges, the Banking Management System was developed to provide a secure, organized, and efficient solution for handling banking operations. The computerized system reduces paperwork, improves data accuracy and security, speeds up transaction processing, and simplifies record management. It also minimizes the risk of manual errors, safeguards sensitive customer information from unauthorized access, and allows for quick and reliable retrieval of records whenever needed. As a result, the system enhances overall banking operations and provides faster, more secure, and more dependable services for both customers and employees, marking a significant improvement over the traditional manual approach.

### **3. Objectives**

The main objective of this project is to develop a Simple Banking System that facilitates efficient and secure management of basic banking operations. The specific objectives achieved through this project are:

- i. To design a database for managing customer and transaction records, and account creation.
- ii. To implement secure user authentication and authorized access, as well as deposit with accurate balance updates and transaction history recording for effective account monitoring effective monitoring transaction.
- iii. To develop a reliable password-reset function that securely handles user credential updates, encrypts the new password, and synchronizes the change with the system files.

## 4. Background Study

In the past, banks managed customer records and transactions manually using paper files and registers. Although this method worked to some extent, it was slow, time-consuming, and prone to human errors, theft, data loss, damage from hazards such as fire or water, and simple forgetfulness on the part of staff. As the number of customers increased, managing records manually became increasingly difficult, since large volumes of paper documents made it harder to retrieve information quickly and ensure the confidentiality of sensitive financial data. This limitation of manual information systems has been widely discussed in *Management Information Systems: Managing the Digital Firm* (Laudon & Laudon, 2022), which highlights how traditional record-keeping methods struggle to scale with growing organizational needs.

With the advancement of technology, computerized banking systems were introduced to improve efficiency, accuracy, and security. Research shows that digital systems reduce errors, protect data through access controls and authentication mechanisms, speed up transactions by automating repetitive tasks, and make information easier to store, organize, and retrieve compared to paper-based methods (Reynolds, n.d.; Pearson Education, n.d.). These sources emphasize that computerization provides a foundation for better decision-making through reliable and readily available data.

However, despite these advancements, many small organizations and educational projects still rely on manual or very basic systems that lack proper security measures and efficient record management. Therefore, this project proposes a Simple Banking System that allows users to create accounts, deposit and withdraw money, check balances, view transaction history, and securely change passwords, all through an easy-to-use interface. The system stores data in a structured file-based format, updates records automatically with every transaction, and improves accuracy, security, and overall banking operations while significantly reducing paperwork and manual effort. This approach follows the principles of computerized information systems discussed by Laudon and Laudon (2022).

## **5. Requirement Document**

A project cannot be implemented without proper requirements being studied. Requirements help communicate and define business needs and problems. The process involves requirements gathering, analysis and documentation, which together act as a compass for the project from beginning to end.

The benefits of preparing a requirement document include:

- Alignment: Consensus and agreement among all team members and the supervisor.
- Preparation: More accurate estimates of the time needed for each module.
- Direction: Clear information for design, development and testing work.
- Efficiency: A defined plan before starting design and coding work.
- Productivity: Less rework and prevention of scope creep.

### **5.1 Types of Requirements**

#### **1. Business Requirements**

The system should computerize basic banking operations so that account and transaction records are accurate, secure and permanently stored, reducing the paperwork and errors of a manual process.

#### **2. User Requirements**

Users want to create a personal account with a unique username, log in securely, deposit money, view their current balance and past transactions, change their password when needed and delete their account if they no longer wish to use the system.

#### **3. Functional Requirements**

The functional (must-have) requirements of the system are: sign up with duplicate-username checking, login authentication, deposit with amount validation, balance enquiry, transaction history listing, password change with verification of the current password, account deletion with confirmation, and automatic saving and loading of all data to and from the file bankdata.txt.

#### 4. Non-Functional Requirements

The non-functional requirements identified are: data persistence across program runs, immediate saving after every change so that data is not lost on unexpected exit, simple menu-driven usability, and low resource usage so the program runs on any computer with a C compiler.

#### 5.2 Requirement Matrix:

The requirements were prioritized using the following requirements matrix:

| S.N. | Required Module / System            | Description                                                                                                       | Priority | Remarks                             |
|------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------|
| 1    | Sign Up and Login (security system) | Only registered users with a valid username and password can access an account; duplicate usernames are rejected. | High     | Khagaraj looked after this feature. |
| 2    | Add Balance (deposit)               | Deposit amount must be validated (greater than zero), added to the balance and recorded in history.               | High     | Ankit looked after this feature.    |
| 3    | Check Balance and View History      | Display the current balance and the numbered list of all past deposits.                                           | High     | Ankit looked after this feature.    |
| 4    | Change Password                     | Current password must be verified before the new password is saved.                                               | Moderate | Anish team worked together.         |
| 5    | Delete Account                      | Y/N confirmation before the record is removed from the file.                                                      | Moderate | Anish team worked together.         |
| 6    | File storage (saveData / loadData)  | All records are saved to bankdata.txt after every change and loaded at program start.                             | High     | Khagaraj team worked together.      |

Table 5.1:Requirement Matrix

## 6. System Design

### 6.1 Gantt Chart

A Gantt chart is a project management tool that displays tasks on a timeline, showing their start and end dates. It helps track project progress, schedule activities, and manage task dependencies efficiently.

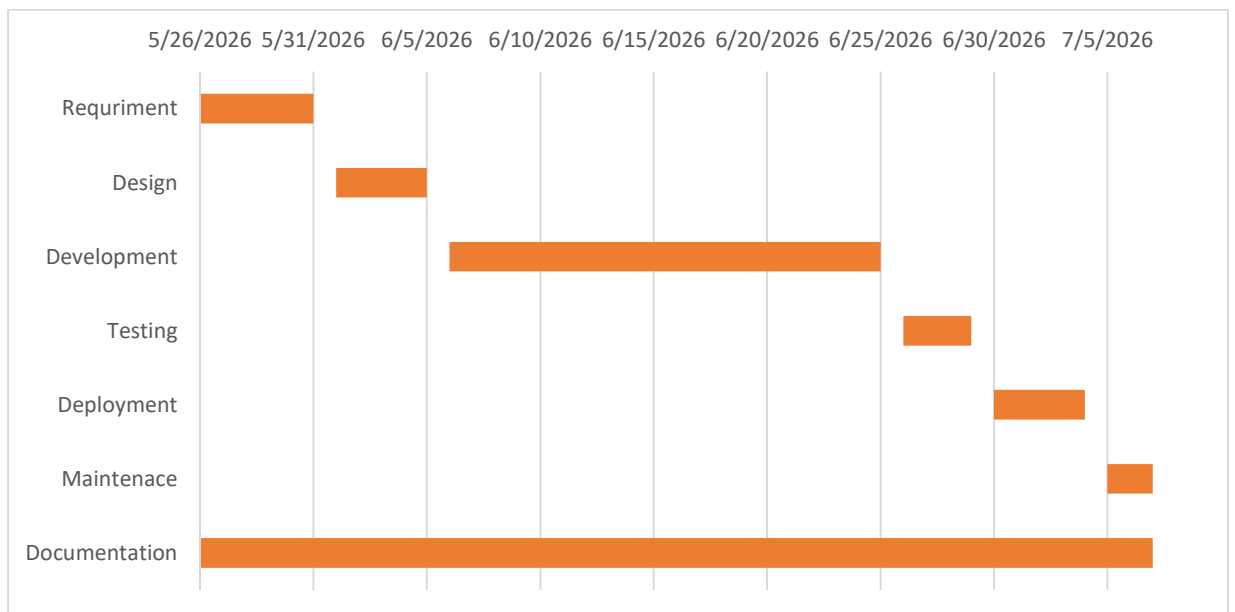


Figure 1: Gantt Chart

## 6.2 Use Case Diagram

The use case diagram defines the behavioural boundaries of the application, illustrating how the primary actor (user) triggers core operations without detailing the background storage mechanics.

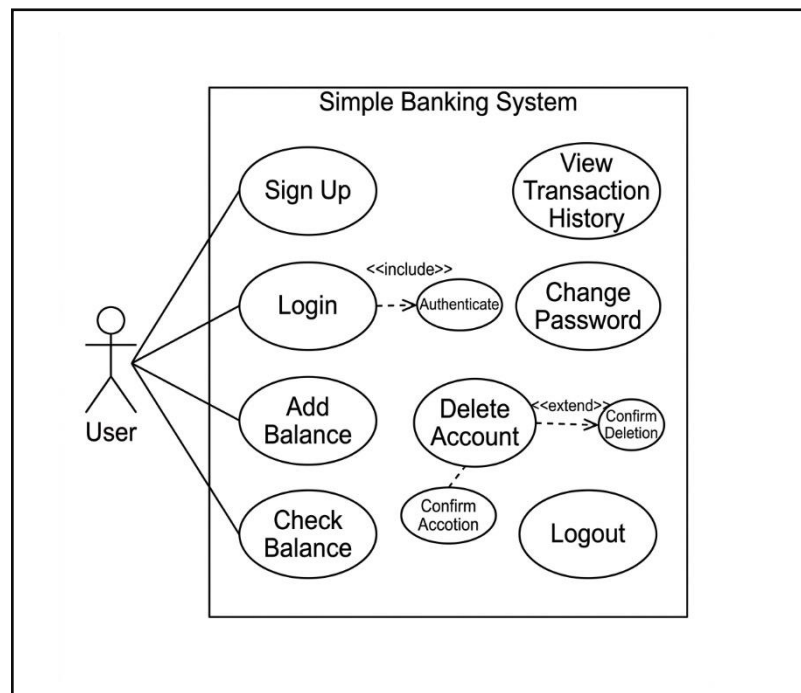


Figure 2: Use Case Diagram of Banking System

### 6.3 Data Flow Diagram

A Data Flow Diagram (DFD) Level 0, also known as the Context Diagram, provides a high-level overview of the entire system by representing it as a single process. It illustrates the interaction between the system and external entities through data flows but does not show any internal processes or data stores

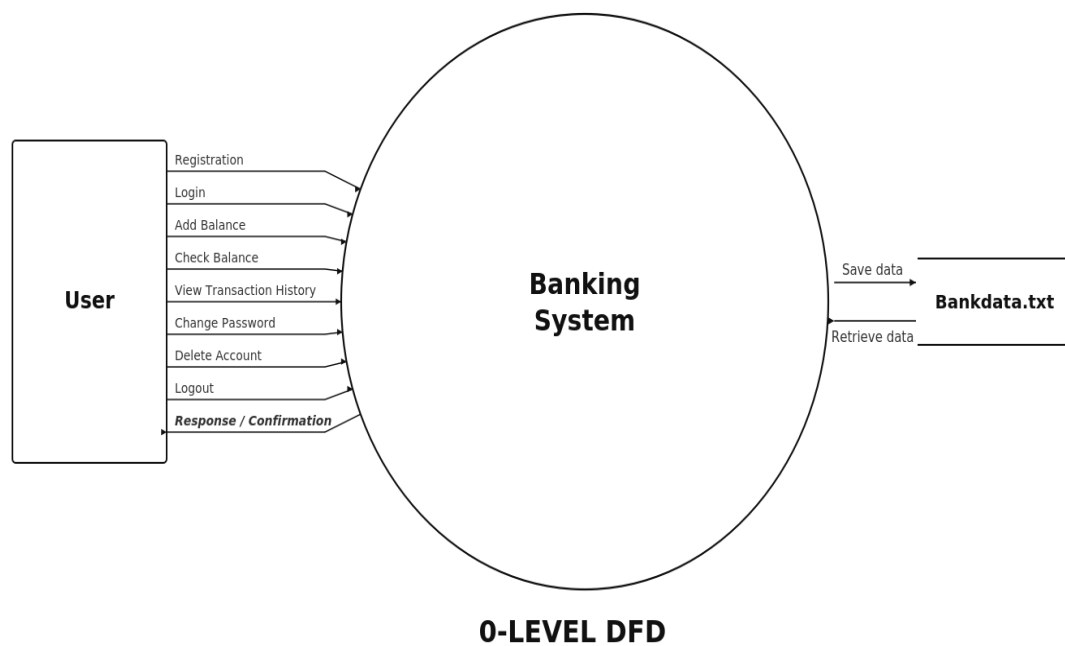


Figure 3: Data Flow Diagram Level 0

A Level 1 Data Flow Diagram (DFD) is a more detailed version of a Level 0 DFD. It breaks down the main processes shown in the Level 0 diagram into smaller sub-processes, illustrating how data moves between processes, data stores, and external entities. A Level 1 DFD provides a clearer understanding of the system's internal operations while maintaining the same overall inputs and outputs as the Level 0 DFD.

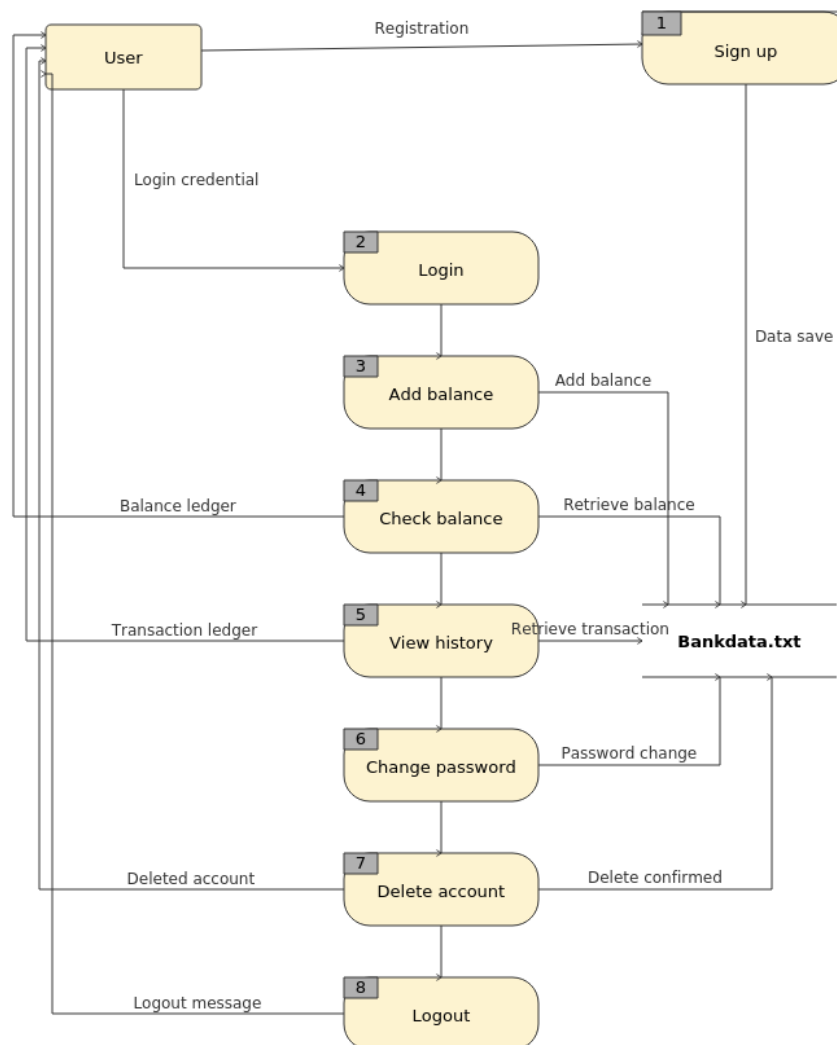


Figure 4: Data Flow Diagram Level 1

## 6.4 Flowchart

A flowchart is a graphical representation of a process, algorithm, or workflow using standardized symbols connected by arrows.

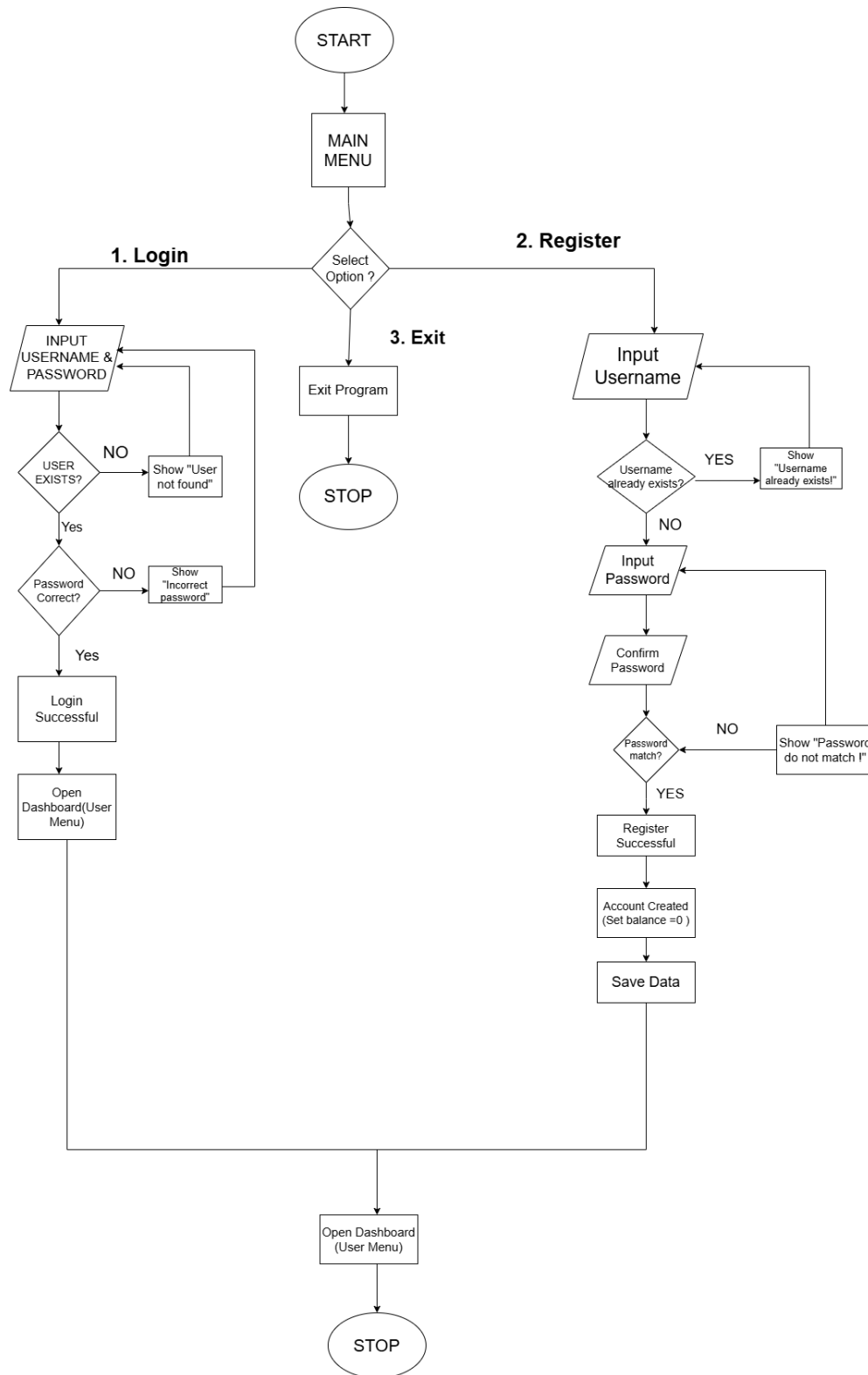


Figure 5: Flowchart of Simple Banking

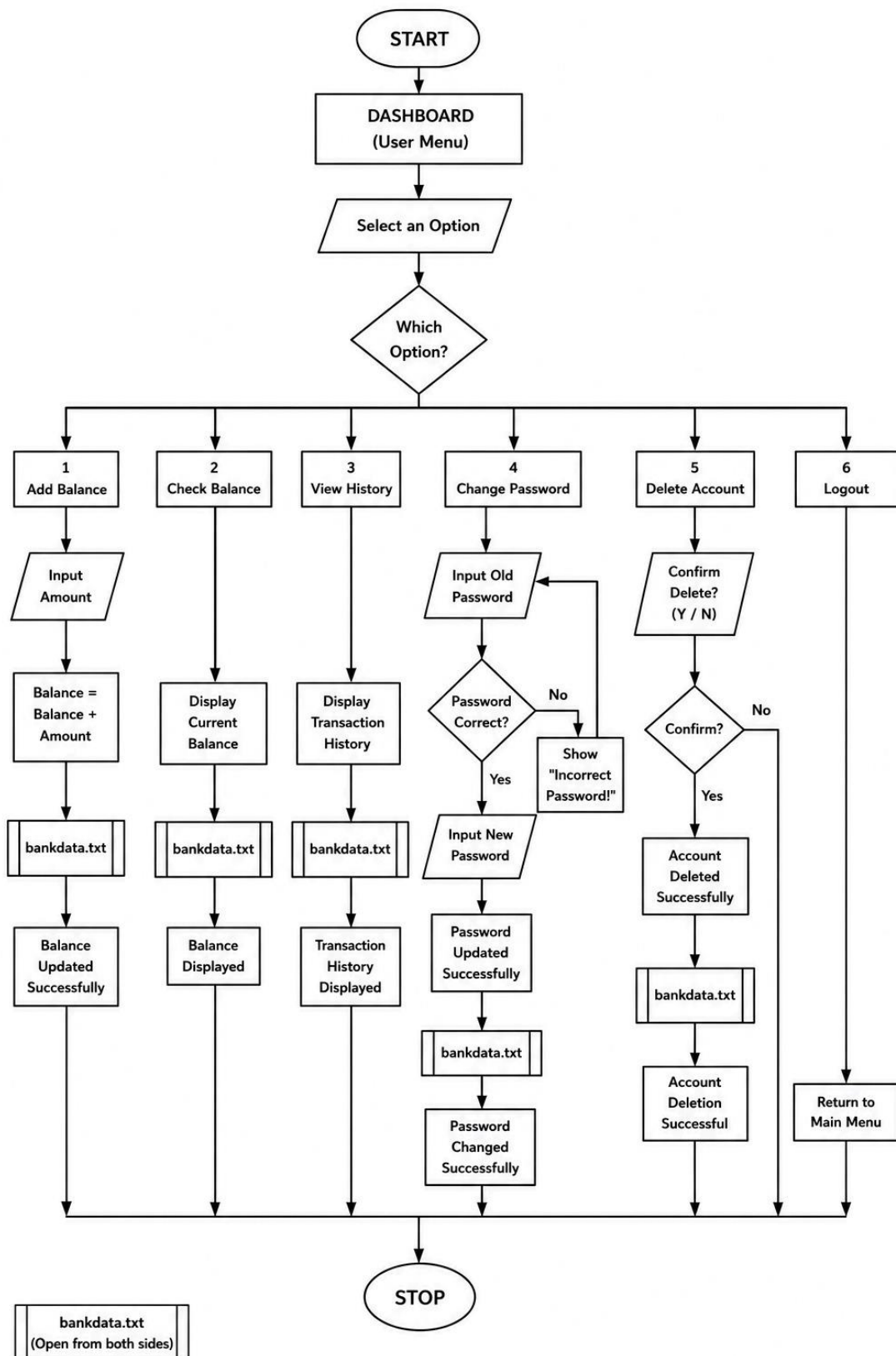


Figure 6: Flowchart of Dashboard

## 7. Development

Development refers to the model used for the project as well as the overall process of planning, organizing, coordinating and controlling the resources to accomplish the project objectives.

### 7.1 Development Model

The project was developed using the waterfall model, because the requirements of the system were small, fixed and clearly understood at the beginning. The work proceeded through the sequential phases of requirement analysis, system design, implementation (coding), testing and documentation. Each phase was completed and reviewed before moving to the next, which suited a first project with a short timeline.

### 7.2 Tools and Collaboration Platform

The system was written in the C programming language and compiled with the GCC compiler using the Code: Blocks IDE. The team used GitHub to keep a shared copy of the source code and track changes, Google Drive to share the documentation files, and Messenger for day-to-day communication and coordination of the work. These platforms were chosen because they are free, simple to use and allowed every member to always work on the latest version of the code and the report.

### 7.3 Work Assignment

The chart of work assignment among the team members is given below:

| S.N. | Name of the Member | Work Assigned                                                                    | Remarks                          |
|------|--------------------|----------------------------------------------------------------------------------|----------------------------------|
| 1    | Khagaraj Poudel    | Documentation and coding: sign up, login and file handling (saveData, loadData). | Also prepared the DFD.           |
| 2    | Ankit Sharma       | Documentation and coding: add balance, check balance and transaction history.    | Also prepared the flowcharts.    |
| 3    | Anish Giri         | Documentation and coding: change password, delete account and menu design.       | Also carried out module testing. |

Table 7.1: Work Assignment Table

## 8. Testing

Testing was carried out following the Software Testing Life Cycle (STLC), which is an integral part of the Software Development Life Cycle (SDLC) but deals only with the testing phases. STLC starts as soon as the requirements are defined and provides a step-by-step process to ensure quality software. The main motive behind testing is to understand the requirements, verify that every module behaves as expected and find defects before the system is delivered.

The six major phases of STLC followed in this project were:

- Requirement Analysis – the requirement document was studied to identify what must be tested.
- Test Planning – the team planned module-wise (forward) testing of each function after it was coded.
- Test Case Designing – test cases were created for each module based on the requirements.
- Test Environment Setup – the compiled program and a sample bankdata.txt file were prepared.
- Test Execution – the test cases were run and defects were recorded and fixed.
- Test Closure – the results were documented in the test matrix and traceability matrix below.

## 8.1 Example of a Test Case

Title: Module testing of the Login function Authentication of users

TID: TC02, TC03

Description: A user should be able to log in to the Simple Banking System only when a valid, previously registered username and password are entered. The system must reject any incorrect or improperly formatted credentials.

Precondition: The user must already have an account registered through the Sign Up module, and the corresponding record must exist in bankdata.txt.

Test Steps: (1) Open the executable file of the project and navigate to the Login option from the main menu. (2) Enter a registered username with an incorrect password (TC02) and observe the system's response. (3) Enter random, unregistered strings as username and password and observe the response. (4) Enter the correct username with different capitalisation to check whether the system treats it as case-sensitive. (5) Enter the correct, valid username and password exactly as registered (TC03) and observe the response.

Expected Result: For TC02, the system displays an "Invalid username or password" message and does not grant access. For TC03, the system accepts the credentials, login succeeds, and the user menu is opened.

Actual Result: The system behaved as expected for both cases – invalid credentials were rejected with the correct error message, and valid credentials successfully opened the user menu.

Status: Pass, consistent with Table 8.1 and the Requirement Traceability Matrix (Table 8.2), where requirement 102 – "Login into the application" – is traced to TC02 and TC03, both marked Pass.

## 8.2 Test Matrix of Tested Modules

| TID  | Module          | Test Description                                        | Expected Result                                       | Status |
|------|-----------------|---------------------------------------------------------|-------------------------------------------------------|--------|
| TC01 | Sign Up         | Create a new account with an already existing username. | “Username already exists” message is shown.           | Pass   |
| TC02 | Login           | Log in with a wrong password.                           | “Invalid username or password” message is shown.      | Pass   |
| TC03 | Login           | Log in with correct username and password.              | Login succeeds and the user menu opens.               | Pass   |
| TC04 | Add Balance     | Deposit a negative amount or zero.                      | “Invalid amount” message; balance unchanged.          | Pass   |
| TC05 | Add Balance     | Deposit a valid amount.                                 | Balance increases and the deposit appears in history. | Pass   |
| TC06 | Check Balance   | View balance after a deposit.                           | The updated balance is displayed.                     | Pass   |
| TC07 | View History    | View history with no transactions.                      | “No transactions available” message is shown.         | Pass   |
| TC08 | Change Password | Enter a wrong current password.                         | “Incorrect password” message; password unchanged.     | Pass   |
| TC09 | Delete Account  | Cancel deletion by entering N.                          | Account is not deleted and menu is shown again.       | Pass   |
| TC10 | File Storage    | Restart the program after a deposit.                    | The saved balance and history are loaded correctly.   | Pass   |

Table 8.1: Module wise Test Cases

### 8.3 Requirement Traceability Matrix

The Requirement Traceability Matrix (RTM) maps and traces each user requirement to its test cases, collecting all requirements and their test status in a single document:

| S.N. | Requirement Description               | Test Case ID | Status                      | Remarks              |
|------|---------------------------------------|--------------|-----------------------------|----------------------|
| 101  | Account creation with unique username | TC01         | TC01 – Pass                 | No changes required. |
| 102  | Login into the application            | TC02, TC03   | TC02 – Pass,<br>TC03 – Pass | No changes required. |
| 103  | Deposit with validation and history   | TC04, TC05   | TC04 – Pass,<br>TC05 – Pass | No changes required. |
| 104  | Balance enquiry and history view      | TC06, TC07   | TC06 – Pass,<br>TC07 – Pass | No changes required. |
| 105  | Password change with verification     | TC08         | TC08 – Pass                 | No changes required. |
| 106  | Account deletion and data persistence | TC09, TC10   | TC09 – Pass,<br>TC10 – Pass | No changes required. |

Table 8.2: Requirement Traceability Matrix

### 8.4 User Acceptance Testing

User Acceptance Testing (UAT), also known as beta or end-user testing, is testing the software by the user or client to determine whether it can be accepted or not. Fellow students were asked to use the system as end users; they were able to create accounts, log in, deposit money and view their history without guidance, and they accepted the system as easy to use.

Supervisor's note: the system works well for its intended scope, and further updates such as withdrawals, password encryption and a graphical interface are recommended as future improvements.

## 9. Project Results

Transaction management easy, accurate and permanent. After completing development and testing The Simple Banking System project was conceptualized to make basic account and, the system fulfils all of the objectives set at the beginning of the project.

The problems that have been solved by the project are:

- Time consumed in manual record searching is reduced, since a user's record is found instantly after login.
- Duplicate accounts are prevented, because the system rejects usernames that already exist.
- Data is stored in a proper structured way; customer details and transaction details are quickly and comfortably found.
- Data can be copied simply and stored in multiple places, which lowers the risk of data loss.
- Calculation errors are eliminated, as the balance is updated automatically after every validated deposit.
- Clarity and consistency in operation is achieved through the fixed menu-driven interface.

At the beginning of the project, we were unfamiliar with C programming and were apprehensive about the project risks. Through this project, we learned structured programming, file handling, debugging, testing and teamwork, and gained the confidence to take on larger projects in the coming semesters.

## 10. Conclusion

The Simple Banking System successfully computerizes the fundamental operations of a bank account: registration, authentication, deposits, withdrawals, balance enquiry, transaction history, password management, and account deletion. The problem of unreliable and error-prone manual record-keeping identified in the problem statement was effectively addressed through structured file-based storage, in which every change made by the user is immediately saved to bankdata.txt and reloaded automatically at the next program start, ensuring that no data is lost between sessions and that the system remains consistent even after unexpected shutdowns.

The system was designed using flowcharts and a Level-1 Data Flow Diagram to clearly map out the logic and flow of information before any coding began, which helped the team visualize the interactions between different modules and identify potential issues early in the development process. Development followed the waterfall model, with a clear division of work among team members across the requirement analysis, design, coding, and testing phases, allowing each stage to be completed, reviewed, and approved before moving to the next. This structured approach minimized rework and ensured that the final product remained aligned with the originally defined requirements throughout the development cycle.

The system was thoroughly verified through module-wise testing, with each test case traced back to its corresponding requirement in the Requirement Traceability Matrix, ensuring complete requirement coverage and accountability. All test cases passed successfully, confirming that the implementation met the specified functional and non-functional requirements. User Acceptance Testing further validated the system's usability, confirming that it is simple and intuitive enough for a new user to operate confidently without any prior guidance or training, which reflects positively on the clarity of the interface design and the logical flow of operations.

Beyond meeting its technical objectives, the project also served as a valuable learning experience for the team. It provided hands-on exposure to the complete software development lifecycle — from requirement gathering and system design through coding, testing, and documentation — while also reinforcing important professional skills such as teamwork, task coordination, effective communication, and adherence to

project timelines. Working under the waterfall model also taught the team the importance of careful upfront planning, since any changes to requirements after the design phase could potentially disrupt the entire schedule.

Overall, the Simple Banking System stands as a reliable, user-friendly, and well-documented solution that fulfills all of its stated objectives and lays a solid foundation for potential future enhancements, such as multi-user support, interest calculation, fund transfers between accounts, or a graphical user interface. The knowledge and experience gained through this project will undoubtedly prove valuable as the team members progress toward more advanced and complex software development endeavors in the future.

## 11. References

- i. K.C, L. (n.d.). *Management Information System*.
- ii. Laudon, L. a. (2022). *Managing the Digital Firm*. Retrieved from <https://www.pearson.com/en-us/subject-catalog/p/management-information-systems/managing-the-digital-firm>
- iii. Reynolds, S. &. (n.d.). *Principles of Information System*. Retrieved from <https://www.cengage.com/c/principles-of-information-systems>

## 12. Annexures

```
===== SIMPLE BANKING SYSTEM =====  
1. Sign Up  
2. Login  
3. Exit  
Enter choice: |
```

Figure 7: Login and Signup

```
Enter username: Saugat  
Enter password: Ankit  
Data saved!  
  
Account created successfully!  
Press any key to continue . . . |
```

Figure 8: Account Creation

```
Enter username: Saugat
Enter password: Ankit

Login successful!
Press any key to continue . . . |
```

Figure 9: Account login

```
===== USER MENU =====
1. Add Balance
2. Check Balance
3. View History
4. Change Password
5. Delete Account
6. Logout
Enter choice: |
```

Figure 10: User Menu

```
Enter amount to add: 2000
Data saved!

Balance updated successfully!
Current Balance: 2000.00
Press any key to continue . . . |
```

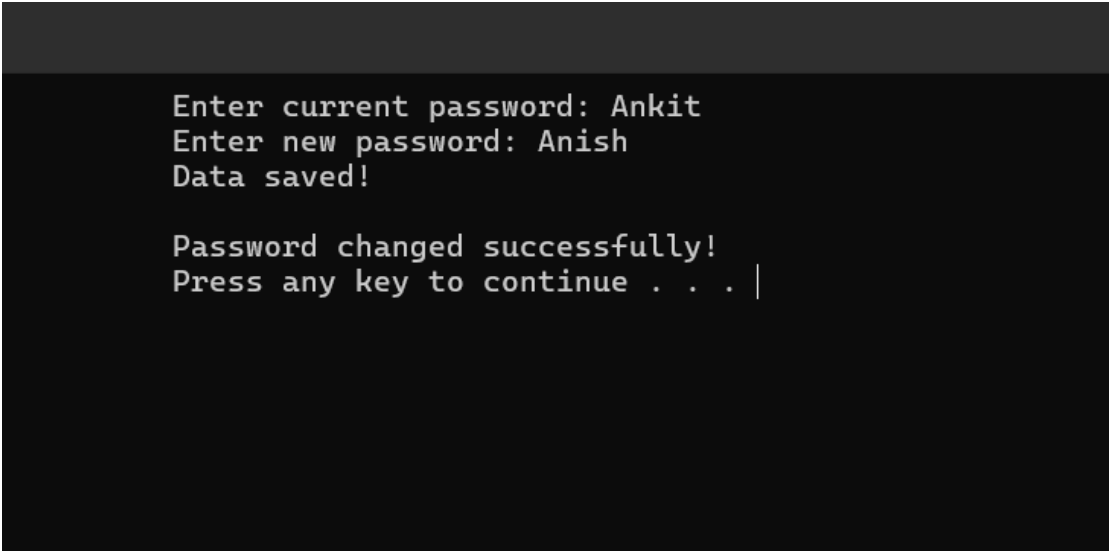
Figure 11: Add balance

```
===== ACCOUNT BALANCE =====
Current Balance: 2000.00
Press any key to continue . . . |
```

Figure 12: Check Balance

```
===== TRANSACTION HISTORY =====
1. Added 2000.00
Press any key to continue . . . |
```

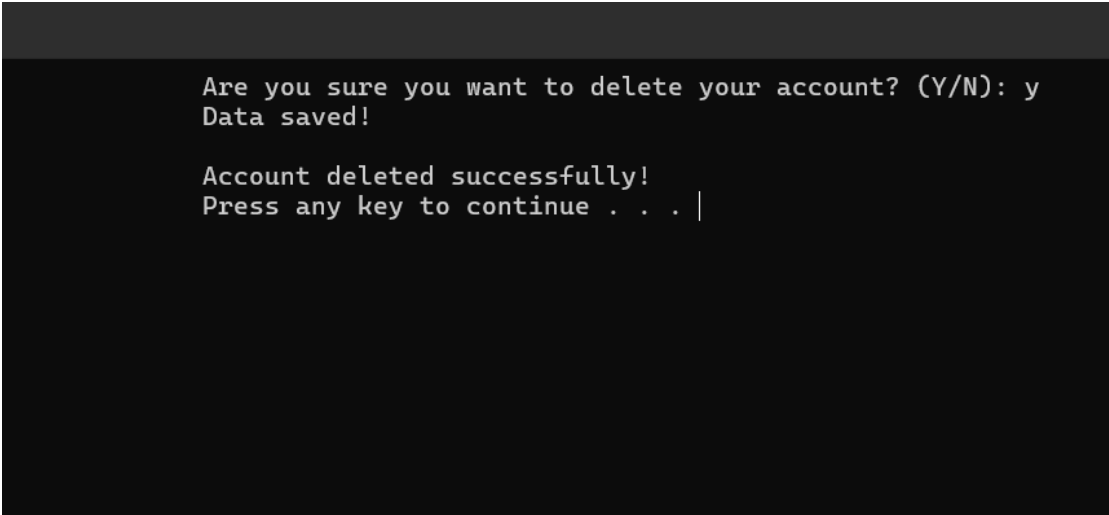
Figure 13: Transaction History

A terminal window with a dark background and a grey title bar. It displays the process of changing a password. The user enters 'Ankit' as the current password and 'Anish' as the new password. The system confirms the data is saved and the password change is successful. A cursor is visible at the end of the final prompt line.

```
Enter current password: Ankit
Enter new password: Anish
Data saved!

Password changed successfully!
Press any key to continue . . . |
```

Figure 14: Change Password

A terminal window with a dark background and a grey title bar. It displays the process of deleting an account. The user is prompted with a confirmation question and responds with 'y'. The system confirms the account deletion and the data is saved. A cursor is visible at the end of the final prompt line.

```
Are you sure you want to delete your account? (Y/N): y
Data saved!

Account deleted successfully!
Press any key to continue . . . |
```

Figure 15: Delete Account

```
===== USER MENU =====  
1. Add Balance  
2. Check Balance  
3. View History  
4. Change Password  
5. Delete Account  
6. Logout  
Enter choice: 6  
  
Logging out...  
Press any key to continue . . . |
```

Figure 16: Account Logout